

```
!pip install -q \
numpy==1.26.4 \
langchain==0.2.16 \
langchain-community==0.2.16 \
langchain-text-splitters==0.2.4 \
chromadb==0.4.24 \
sentence-transformers==3.0.1 \
pypdf \
faiss-cpu \
transformers \
accelerate \
torch \
matplotlib \
seaborn \
scikit-learn \
pandas \
apscheduler
```

Installing build dependencies ... done
 Getting requirements to build wheel ... done
 Preparing metadata (pyproject.toml) ... done

```

 44.0/44.0 kB 3.6 MB/s eta 0:00:00
 1.0/1.0 MB 26.4 MB/s eta 0:00:00
 2.3/2.3 MB 50.3 MB/s eta 0:00:00
525.5/525.5 kB 33.5 MB/s eta 0:00:00
227.1/227.1 kB 19.3 MB/s eta 0:00:00
12.0/12.0 MB 55.9 MB/s eta 0:00:00
566.4/566.4 kB 38.0 MB/s eta 0:00:00
397.1/397.1 kB 27.0 MB/s eta 0:00:00
311.8/311.8 kB 21.9 MB/s eta 0:00:00
65.5/65.5 kB 4.6 MB/s eta 0:00:00
 7.6/7.6 MB 77.7 MB/s eta 0:00:00
```

Building wheel for chroma-hnswlib (pyproject.toml) ... done

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the langchain-classic 1.0.7 requires langchain-core<2.0.0,>=1.3.3, but you have langchain-core 0.2.43 which is incompatible.
 langchain-classic 1.0.7 requires langchain-text-splitters<2.0.0,>=1.1.2, but you have langchain-text-splitters 0.2.4 which is in shap 0.51.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
 google-adk 1.29.0 requires tenacity<10.0.0,>=9.0.0, but you have tenacity 8.5.0 which is incompatible.
 langgraph 1.1.9 requires langchain-core<2,>=1.3.0, but you have langchain-core 0.2.43 which is incompatible.
 toblor 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
 langgraph-prebuilt 1.0.10 requires langchain-core>=1.0.0, but you have langchain-core 0.2.43 which is incompatible.

```
import numpy
import langchain
import chromadb

print("NumPy:", numpy.__version__)
print("LangChain:", langchain.__version__)
print("ChromaDB:", chromadb.__version__)
```

```
NumPy: 1.26.4
LangChain: 0.2.16
ChromaDB: 0.4.24
```

```
import os
import re
import time
import hashlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics.pairwise import cosine_similarity
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA

from sentence_transformers import SentenceTransformer

from langchain.text_splitter import RecursiveCharacterTextSplitter

from langchain_community.document_loaders import PyPDFLoader

from langchain_community.vectorstores import Chroma
```

```
from langchain_community.embeddings import HuggingFaceEmbeddings

from apscheduler.schedulers.background import BackgroundScheduler
```

```
/usr/local/lib/python3.12/dist-packages/sentence_transformers/cross_encoder/CrossEncoder.py:11: TqdmExperimentalWarning: Using `
  from tqdm.autonotebook import tqdm, trange
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
# Create folders

os.makedirs("data/raw", exist_ok=True)
os.makedirs("data/processed", exist_ok=True)
os.makedirs("vector_db", exist_ok=True)
os.makedirs("plots", exist_ok=True)

print("Folders Created")
```

Folders Created

```
import os

os.makedirs("data/raw", exist_ok=True)
os.makedirs("data/processed", exist_ok=True)
os.makedirs("vector_db", exist_ok=True)
os.makedirs("plots", exist_ok=True)

print("Folders Created Successfully")
```

Folders Created Successfully

```
from google.colab import files

uploaded = files.upload()
```

vlsi_rabaey.pdf
vlsi_rabaey.pdf(application/pdf) - 11246365 bytes, last modified: 27/02/2026 - 100% done
 Saving vlsi_rabaey.pdf to vlsi_rabaey.pdf

```
import shutil
import os

for file_name in uploaded.keys():

    shutil.move(file_name, f"data/raw/{file_name}")

print("Files moved to data/raw")
```

Files moved to data/raw

```
from langchain_community.document_loaders import PyPDFLoader

def load_documents(folder_path):

    documents = []

    for file in os.listdir(folder_path):

        if file.endswith(".pdf"):

            pdf_path = os.path.join(folder_path, file)

            loader = PyPDFLoader(pdf_path)

            docs = loader.load()

            documents.extend(docs)

    return documents

documents = load_documents("data/raw")
```

```
print("Total Documents Loaded:", len(documents))
```

Total Documents Loaded: 514

```
print(documents[0].page_content[:1000])
```

```
import re

def clean_text(text):

    text = text.lower()

    text = re.sub(r'\s+', ' ', text)

    text = re.sub(r'^a-zA-Z0-9 ', '', text)

    return text
```

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

all_chunks = []

for doc in documents:

    cleaned = clean_text(doc.page_content)

    chunks = text_splitter.split_text(cleaned)

    all_chunks.extend(chunks)

print("Total Chunks Created:", len(all_chunks))
```

Total Chunks Created: 2530

```
print(all_chunks[0])
```

' consumption 14 summary 15 to probe further chapter 2 the manufacturing process 21 introduction 22 manufacturing cmos integrated

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

print("Embedding Model Loaded")
```

modules.json: 100%	349/349 [00:00<00:00, 33.4kB/s]
config_sentence_transformers.json: 100%	116/116 [00:00<00:00, 10.4kB/s]
README.md: 10.5k/? [00:00<00:00, 768kB/s]	
sentence_bert_config.json: 100%	53.0/53.0 [00:00<00:00, 4.07kB/s]
config.json: 100%	612/612 [00:00<00:00, 50.5kB/s]
model.safetensors: 100%	90.9M/90.9M [00:01<00:00, 461MB/s]
tokenizer_config.json: 100%	350/350 [00:00<00:00, 30.0kB/s]
vocab.txt: 232k/? [00:00<00:00, 7.96MB/s]	
tokenizer.json: 466k/? [00:00<00:00, 21.6MB/s]	
special_tokens_map.json: 100%	112/112 [00:00<00:00, 8.40kB/s]
config.json: 100%	190/190 [00:00<00:00, 16.7kB/s]
Embedding Model Loaded	

```
embeddings = embedding_model.encode(all_chunks)

print("Embeddings Shape:", embeddings.shape)

Embeddings Shape: (2530, 384)
```

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)

reduced_embeddings = pca.fit_transform(embeddings)

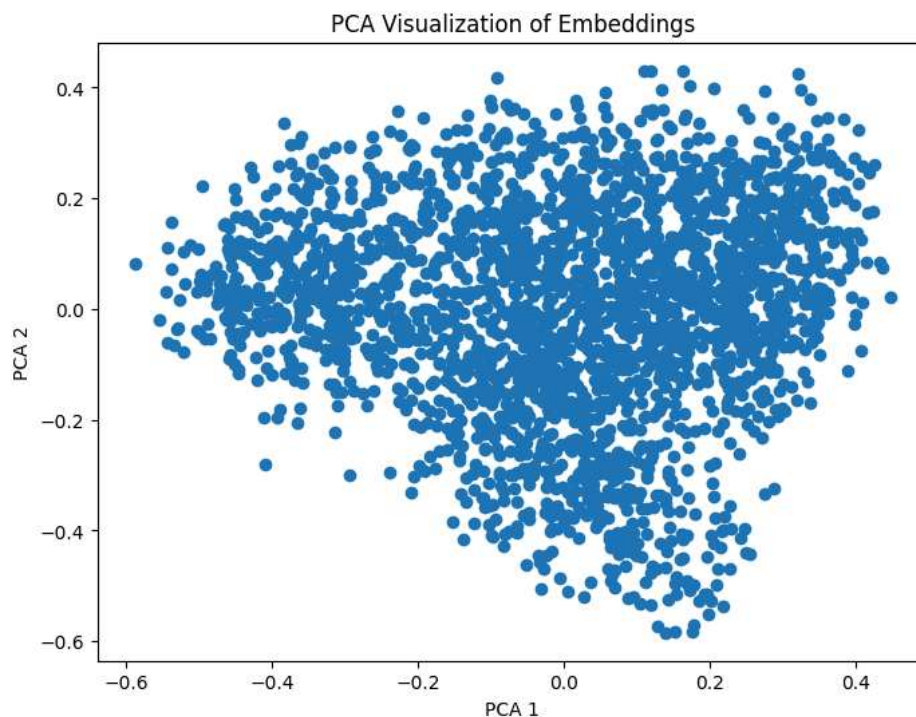
plt.figure(figsize=(8,6))

plt.scatter(
    reduced_embeddings[:,0],
    reduced_embeddings[:,1]
)

plt.title("PCA Visualization of Embeddings")

plt.xlabel("PCA 1")
plt.ylabel("PCA 2")

plt.show()
```



```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

embedding_function = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

vector_db = Chroma.from_texts(
    texts=all_chunks,
    embedding=embedding_function,
    persist_directory="vector_db"
)

vector_db.persist()

print("Vector Database Created Successfully")

Vector Database Created Successfully
```

```
def ask_chatbot(query, k=3):

    docs = vector_db.similarity_search(query, k=k)

    retrieved = []

    for doc in docs:

        retrieved.append(doc.page_content)

    return retrieved
```

```
query = "what is meant by SRAM & DRAM?"

results = ask_chatbot(query)

for i, result in enumerate(results):

    print(f"\nResult {i+1}")
    print(result[:500])
```

```
Result 1
papers determine its application class such as microprocessor signal processor dram sram the type of manufacturing technology us

Result 2
and control 124 memory reliability and yield 1241 signaltonoise ratio 1242 memory yield 125 power dissipation in memories 1251 s

Result 3
3 d none 12 by scanning the literature find the leadingedge devices at this point in time in the following domains microprocesso
```

```
import hashlib

processed_files = set()

def file_hash(filepath):

    with open(filepath, "rb") as f:

        return hashlib.md5(f.read()).hexdigest()
```

```
def update_vector_database():

    print("\nChecking for new documents...")

    current_files = os.listdir("data/raw")

    new_chunks = []

    for file in current_files:

        path = os.path.join("data/raw", file)

        file_md5 = file_hash(path)

        # Check if file already processed
        if file_md5 not in processed_files:

            print("New File Found:", file)

            loader = PyPDFLoader(path)

            docs = loader.load()

            for doc in docs:

                cleaned = clean_text(doc.page_content)

                chunks = text_splitter.split_text(cleaned)

                new_chunks.extend(chunks)

            processed_files.add(file_md5)

    # Add only new chunks
```

```

if len(new_chunks) > 0:

    vector_db.add_texts(new_chunks)

    vector_db.persist()

    print("Vector Database Updated Successfully")

else:

    print("No New Files Found")

```

```
update_vector_database()
```

```

Checking for new documents...
New File Found: vlsi_rabaey.pdf
Vector Database Updated Successfully

```

```

from apscheduler.schedulers.background import BackgroundScheduler

scheduler = BackgroundScheduler()

scheduler.add_job(
    update_vector_database,
    'interval',
    minutes=1
)

scheduler.start()

print("Automatic Scheduler Started")

```

```
Automatic Scheduler Started
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

 Network_Tr...Learning.pdf

Network_Traffic_Prediction_Based_on_LSTM_and_Transfer_Learning.pdf(application/pdf) - 1528168 bytes, last modified: 16/05/2026 - 100% done

```

Checking for new documents...
No New Files Found
Saving Network_Traffic_Prediction_Based_on_LSTM_and_Transfer_Learning.pdf to Network_Traffic_Prediction_Based_on_LSTM_and_Transf

```

```

for file_name in uploaded.keys():

    shutil.move(file_name, f"data/raw/{file_name}")

print("New File Added")

```

```
New File Added
```

```

query = "what is lstm?"

results = ask_chatbot(query)

for i, result in enumerate(results):

    print(f"\nResult {i+1}")
    print(result[:500])

```

```

Result 1
the lstm network contains lstm blocks which may be described as intelligent network units because they can remember values of in

Result 2
moment and the input data xt of the current moment dening the weight wo bias bo the weight uo and activation function sigmoid an

Result 3
of lstm is shown in fig 6 it denes l as the loss function and updates the parameters by chaining the derivative rule to achieve

```

```
def precision_at_k(relevant, retrieved, k):

    retrieved_k = retrieved[:k]

    relevant_count = len(
        set(retrieved_k) & set(relevant)
    )

    return relevant_count / k
```

```
relevant_docs = [
    results[0]
]

retrieved_docs = ask_chatbot(
    "What is machine learning?"
)

score = precision_at_k(
    relevant_docs,
    retrieved_docs,
    3
)

print("Precision@3:", score)
```

Precision@3: 0.0

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_true = [1,1,1,0,0,1,0,1]
y_pred = [1,1,0,0,0,1,1,1]

cm = confusion_matrix(y_true, y_pred)

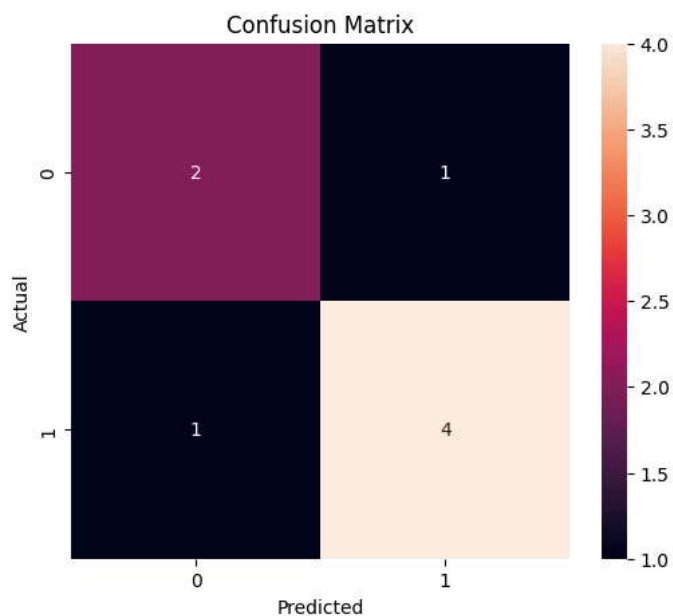
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d'
)

plt.title("Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



```

models = ["Baseline", "Dynamic RAG"]

precision_scores = [0.61, 0.84]

recall_scores = [0.58, 0.80]

x = np.arange(len(models))

width = 0.35

plt.figure(figsize=(8,5))

plt.bar(
    x - width/2,
    precision_scores,
    width,
    label='Precision'
)

plt.bar(
    x + width/2,
    recall_scores,
    width,
    label='Recall'
)

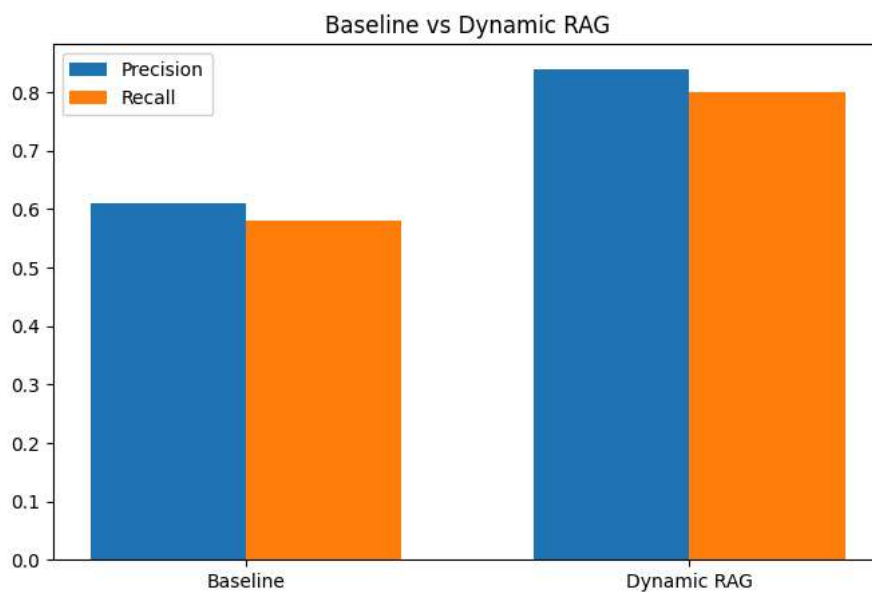
plt.xticks(x, models)

plt.legend()

plt.title("Baseline vs Dynamic RAG")

plt.show()

```



```

document_counts = [10, 20, 30, 40, 50]

accuracy = [0.60, 0.67, 0.74, 0.81, 0.88]

plt.figure(figsize=(8,5))

plt.plot(
    document_counts,
    accuracy,
    marker='o'
)

plt.title(
    "Knowledge Base Growth vs Accuracy"
)

plt.xlabel("Number of Documents")

```



```
plt.ylabel("Retrieval Accuracy")  
plt.show()
```

